

app.js

đây là file `app.js` - file khởi tạo chính của ứng dụng Express.



Giải thích Mã nguồn `app.js`

1. Khai báo và Thiết lập Ban đầu

```
const express=require("express");
const mongoose = require("mongoose");
const app=express();
const path=require("path")
```

- `const express = require("express");` : Tải **framework Express** đã cài đặt vào biến `express`. `require()` là cách chuẩn trong Node.js để **import (nhập khẩu)** một module (thư viện).
- `const app = express();` : Tạo một **instance (thể hiện)** của ứng dụng Express. Biến `app` này là đối tượng chính để cấu hình server (middleware, routing, view engine...).
- `const mongoose = require("mongoose");` : Tải thư viện **Mongoose** để làm việc với MongoDB.
- `const path = require("path");` : Tải module `path` tích hợp sẵn của Node.js. Module này dùng để làm việc với **đường dẫn tệp tin** một cách độc lập với hệ điều hành (ví dụ: `path.join()` đảm bảo dấu gạch chéo `/` hay `\` được sử dụng đúng).

2. Thiết lập Middleware

Middleware là các hàm được thực thi giữa lúc server nhận yêu cầu (request) và lúc gửi phản hồi (response).

A. Static Files Middleware

```
//middleware static uploadFile
app.use(express.static(path.join(__dirname,"public")));
```

- `app.use(...)` : Thêm một hàm middleware vào pipeline xử lý yêu cầu.

- `express.static(...)` : Middleware tích hợp của Express để **phục vụ các tệp tin tĩnh** (static files) như CSS, JavaScript clientside, và hình ảnh.
- `path.join(__dirname, "public")` : Tạo đường dẫn tuyệt đối đến thư mục `public` (ví dụ: `C:\project\session06\public`). Bất kỳ tệp tin nào trong thư mục này đều có thể được truy cập trực tiếp bằng URL (ví dụ: `/style.css`).

B. Body Parser Middleware

```
app.use(express.urlencoded({extended:true}));
```

- `express.urlencoded()` : Đây là một middleware **Body Parser** tích hợp của Express. Chức năng của nó là **phân tích cú pháp (parse)** dữ liệu được gửi từ các biểu mẫu HTML (HTML forms) với header `Content-Type: application/x-www-form-urlencoded` .
- `{extended: true}` : Tùy chọn này cho phép phân tích cú pháp **dữ liệu phức tạp** (ví dụ: đối tượng lồng nhau, mảng) từ biểu mẫu.
- **Tại sao lại dùng `.urlencoded()` ?**
 - Bạn dùng `.urlencoded()` vì nó **chuyên dùng để xử lý dữ liệu gửi từ Form HTML thông thường** (thường dùng phương thức POST).
 - **Có thể dùng cái khác được không? Có.** Express cung cấp các body parser khác tùy thuộc vào định dạng dữ liệu bạn nhận:
 - `express.json()` : Được sử dụng để phân tích cú pháp dữ liệu gửi đến dưới dạng **JSON** (`Content-Type: application/json`). JSON rất phổ biến khi làm việc với API.
 - `multer` : Được sử dụng khi biểu mẫu có chứa các **tệp tin (file upload)** (`Content-Type: multipart/form-data`).

Tóm lại, bạn dùng `.urlencoded()` là **đúng mục đích** nếu ứng dụng của bạn chủ yếu nhận dữ liệu từ các form HTML.

3. Thiết lập View Engine (EJS)

```
//cấu hình view
app.set("views", path.join(__dirname, "views"));
app.set("view engine", "ejs")
```

- `app.set("views", ...)` : Cấu hình Express biết nơi để tìm kiếm các **tệp tin View (giao diện)**. Ở đây là thư mục `views` .

- `app.set("view engine", "ejs")` : Chỉ định **EJS** (Embedded JavaScript) là **View Engine** (công cụ kết xuất giao diện) mặc định. Điều này cho phép bạn sử dụng hàm `res.render('ten_file')` trong router mà không cần chỉ định đuôi file (`.ejs`).

4. Định tuyến (Routing)

```
const rootRouter = require("./routers/rootRouter");
//router
app.use("/", rootRouter); // rootRouter phải là function (router)
```

- `const rootRouter = require("./routers/rootRouter");` : Tải module **Router** đã được định nghĩa trong file `routers/rootRouter.js`.
- `app.use("/", rootRouter);` : Đây là cách để **gắn Router** vào ứng dụng.
 - `"/"` : Là đường dẫn cơ sở (base path). Mọi đường dẫn được định nghĩa trong `rootRouter` sẽ được thêm vào sau `/`.
 - **Lưu ý:** Đúng như comment của bạn, `rootRouter` **phải là một đối tượng Router** được tạo bằng `express.Router()` và được `module.exports` từ file `rootRouter.js`. Cú pháp này là chuẩn mực trong Express.

5. Kết nối Cơ sở Dữ liệu

```
//liên kết mongodb
mongoose.connect("mongodb://localhost:27017/prodb")
.then(() => console.log("connect thanh cong"))
.catch(err => console.log("connect that bai", err))
```

- `mongoose.connect(...)` : Thiết lập kết nối đến máy chủ MongoDB.
- **Chuỗi kết nối (Connection String):** `"mongodb://localhost:27017/prodb"`
 - `mongodb://` : Giao thức.
 - `localhost:27017` : Địa chỉ và cổng mặc định của MongoDB (nếu chạy local).
 - `/prodb` : Tên cơ sở dữ liệu (Database) mà ứng dụng sẽ sử dụng.
- `then()` / `catch()` : Xử lý kết quả kết nối. In ra thông báo thành công hoặc lỗi (nếu thất bại).

6. Khởi chạy Server

```
const PORT=3000;  
app.listen(PORT,()=>{  
  console.log(`Server is running in http://localhost:${PORT}`);  
})
```

- **app.listen(PORT, ...)** : Khởi động server HTTP. Nó lắng nghe các yêu cầu đến trên cổng được chỉ định (ở đây là **3000**).
 - **Hàm Callback**: Hàm mũi tên **() => { ... }** sẽ được thực thi **sau khi server khởi động thành công**. Nó in ra thông báo cho biết địa chỉ truy cập server.
-